

You are required to deploy a Flask backend and an Express frontend application using **AWS** and **Terraform**. Follow the steps below to deploy the applications in different configurations.

Part 1: Deploy Both Flask and Express on a Single EC2 Instance

1. **Objective:** Deploy both the Flask backend and the Express frontend on a **single EC2 instance** using Terraform.
 - Provision the EC2 instance with Terraform.
 - Configure the instance with:
 - A user data script or a configuration management tool (e.g., Ansible or Cloud-Init) to install dependencies (Python, Node.js).
 - Scripts to start the Flask backend and the Express frontend.
 - Ensure both applications are running on different ports (e.g., Flask on port 5000 and Express on port 3000).
 2. **Expected Deliverables:**
 - Terraform configuration files (`main.tf`, `variables.tf`, etc.).
 - A working EC2 instance with Flask and Express running and accessible via the instance's public IP.
-

CODE IS PLACED IN GIT LOCATION URL:

[UmeshChandra2001/terraform-flask-express: TERRAFORM ASSGINMENT -1](https://github.com/UmeshChandra2001/terraform-flask-express)

FOLDER STUCTURE

terraform-flask-express/

|

|— main.tf

|— variables.tf

|— outputs.tf

|— provider.tf

|— terraform.tfvars

|

|— app/

| |— backend/

```
| | |— app.py
| | |— requirements.txt
| |
| |— frontend/
| |— package.json
| |— server.js
|
|— user-data.sh
```

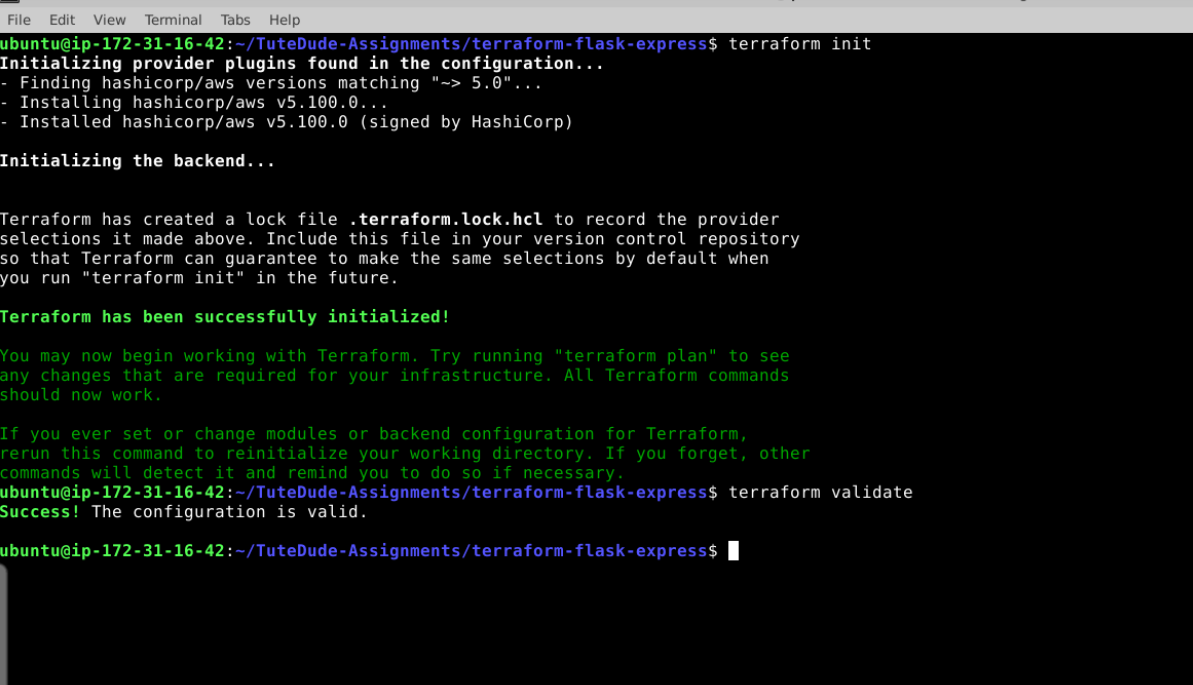
COMMANDS :

Initialize Terraform

CMD : **terraform init**

Validate

CMD : **terraform validate**



```
File Edit View Terminal Tabs Help
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$ terraform validate
Success! The configuration is valid.

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$
```

PREVIEW

CMD : **terraform plan**

```

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$ terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
  + ami                    = "ami-0f58b397bc5c1f2e8"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count         = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop       = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6     = (known after apply)
  + get_password_data      = false
  + host_id                = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile    = (known after apply)
  + id                     = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "t2.micro"
  + ipv6_address_count     = (known after apply)
  + ipv6_addresses        = (known after apply)
  + key_name               = "your-keypair-name"
  + monitoring             = (known after apply)
  + outpost_arn            = (known after apply)
  + password_data          = (known after apply)
  + placement_group        = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns            = (known after apply)
  + private_ip             = (known after apply)
  + public_dns             = (known after apply)
  + public_ip              = (known after apply)
  + secondary_private_ips  = (known after apply)
  + security_groups        = [
    + "flask-express-sg",
  ]
  + source_dest_check      = true
  + spot_instance_request_id = (known after apply)
  + subnet_id              = (known after apply)
  + tags                   = {
    + "Name" = "Flask-Express-Server"
  }
  + tags_all               = {
}

```

DEPLOYING:

CMD : terraform apply -auto-approve

```

Terminal - ubuntu@ip-172-31-16-42: ~/TuteDude-Assignments/terraform-flask-express
File Edit View Terminal Tabs Help

+ {
+   {
+     cidr_blocks = [
+       "0.0.0.0/0",
+     ]
+     from_port    = 3000
+     ipv6_cidr_blocks = []
+     prefix_list_ids = []
+     protocol     = "tcp"
+     security_groups = []
+     self         = false
+     to_port      = 3000
+     # [1 unchanged attribute hidden]
+   },
+   {
+     cidr_blocks = [
+       "0.0.0.0/0",
+     ]
+     from_port    = 5000
+     ipv6_cidr_blocks = []
+     prefix_list_ids = []
+     protocol     = "tcp"
+     security_groups = []
+     self         = false
+     to_port      = 5000
+     # [1 unchanged attribute hidden]
+   },
+   {
+     name           = "flask-express-sg"
+     name_prefix    = (known after apply)
+     owner_id       = (known after apply)
+     revoke_rules_on_delete = false
+     tags_all       = (known after apply)
+     vpc_id         = (known after apply)
+   }
+ }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ public_ip = (known after apply)
aws_security_group.web_sg: Creating...
aws_instance.web_server: Creation complete after 2s [id=sg-0c0967e9b57df8269]
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 12s [id=i-08511bf0196c52a3f]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:
public_ip = "43.204.19.234"
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$ terraform apply -auto-approve

```

Instances (5) [Info](#)

Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets
Choose filter set [Find Instance by attribute or tag \(case-sensitive\)](#)

☐	Name 🔗	Instance ID	Instance state ▲	Instance type ▼	Status check	Alarm status	Avz
☐	UMESH_RDP1	i-09bb46bac5cf14196	Running 🔍 🔍	m7i-flex.large	3/3 checks passed	View alarms +	ap-
☐	Flask-Express-Server	i-08511bf0196c52a3f	Running 🔍 🔍	t3.micro	3/3 checks passed	View alarms +	ap-

To get public IP:

CMD : **terraform output**

```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-flask-express$ terraform output
public_ip = "43.204.19.234"
```

OUTPUT:



Verify Services:

`sudo systemctl status flask`

`sudo systemctl status express`

```
May 10 11:07:33 ip-172-31-37-104 node[8168]: Frontend running on port 3000
ubuntu@ip-172-31-37-104:~$ sudo systemctl status express
● express.service - Express Frontend
   Loaded: loaded (/etc/systemd/system/express.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-10 11:07:32 UTC; 16min ago
  Invocation: 5dc68dff1f39421581b2e1187911f9ea
    Main PID: 8168 (node)
      Tasks: 7 (limit: 627)
     Memory: 21.3M (peak: 25.4M)
        CPU: 567ms
    CGroup: /system.slice/express.service
            └─8168 /usr/bin/node /home/ubuntu/frontend/server.js

May 10 11:07:32 ip-172-31-37-104 systemd[1]: Started express.service - Express Frontend.
May 10 11:07:33 ip-172-31-37-104 node[8168]: Frontend running on port 3000
ubuntu@ip-172-31-37-104:~$ sudo systemctl status flask
● flask.service - Flask Backend
   Loaded: loaded (/etc/systemd/system/flask.service; enabled; preset: enabled)
   Active: failed (Result: exit-code) since Sun 2026-05-10 11:07:35 UTC; 16min ago
  Duration: 158ms
  Invocation: ea25fe7d5cd54f00bfb613aca2ec98f
    Main PID: 8207 (code=exited, status=1/FAILURE)
     Mem peak: 16.7M
        CPU: 154ms

May 10 11:07:35 ip-172-31-37-104 systemd[1]: flask.service: Scheduled restart job, restart counter is at 5.
May 10 11:07:35 ip-172-31-37-104 systemd[1]: flask.service: Start request repeated too quickly.
May 10 11:07:35 ip-172-31-37-104 systemd[1]: flask.service: Failed with result 'exit-code'.
May 10 11:07:35 ip-172-31-37-104 systemd[1]: Failed to start flask.service - Flask Backend.
ubuntu@ip-172-31-37-104:~$
```

Part 2: Deploy Flask and Express on Separate EC2 Instances

1. **Objective:** Deploy the Flask backend and the Express frontend on **two separate EC2 instances** using Terraform.
 - Provision two EC2 instances using Terraform:
 - One for the Flask backend.
 - One for the Express frontend.
 - Configure security groups to:
 - Allow communication between the two instances.
 - Expose both applications to the internet on their respective ports.
 2. **Configuration:**
 - Use Terraform to define networking resources, such as VPC, subnets, and security groups.
 - Use user data scripts to automate the installation and startup of both applications.
 3. **Expected Deliverables:**
 - Terraform configuration files.
 - Two working EC2 instances: one running Flask and one running Express.
 - Security groups configured to allow proper communication and public access.
-

CODE IS PLACED IN GIT LOCATION URL:

[UmeshChandra2001/terraform-ass-part-2: TERRAFORM ASSGINMENT -2](https://github.com/UmeshChandra2001/terraform-ass-part-2)

Project Structure

terraform-ass-part-2/

```
|
|
|— main.tf
|— variables.tf
|— outputs.tf
|— provider.tf
|— terraform.tfvars
|
|— flask-userdata.sh
|— express-userdata.sh
```

CODE DETAILS HAVE SHARED OVER GIT RESPOSITORY

Initialize Terraform

CMD : terraform init

```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$ terraform validate
```

Validate Terraform

CMD : terraform validate

Deploy Infrastructure

CMD : terraform apply -auto-approve

```
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$ terraform validate
Success! The configuration is valid.

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.express_server will be created
+ resource "aws_instance" "express_server" {
+   ami                  = "ami-07a00cf47dbbc844c"
+   arn                  = (known after apply)
+   associate_public_ip_address = true
+   availability_zone     = (known after apply)
+   cpu_core_count        = (known after apply)
+   cpu_threads_per_core  = (known after apply)
+   disable_api_stop      = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized         = (known after apply)
+   enable_primary_ipv6    = (known after apply)
+   get_password_data      = false
+   host_id               = (known after apply)
+   host_resource_group_arn = (known after apply)
}
```

```

+ user_data                                     = fccad4472f8cc8ee5c1d8a46c00a1e7ae4106e1
+ user_data_base64                             = (known after apply)
+ user_data_replace_on_change                  = false
+ vpc_security_group_ids                       = [
+   "sg-0cef0bb3f5fd93ea7",
+ ]

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ express_public_ip = (known after apply)
+ flask_public_ip   = (known after apply)
aws_instance.flask_server: Creating...
aws_instance.flask_server: Still creating... [00m10s elapsed]
aws_instance.flask_server: Creation complete after 12s [id=i-0c7532ed1e41586d5]
aws_instance.express_server: Creating...
aws_instance.express_server: Still creating... [00m10s elapsed]
aws_instance.express_server: Creation complete after 12s [id=i-06baafbb317193147]

```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```

express_public_ip = "15.207.248.203"
flask_public_ip   = "3.108.219.115"

```

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2\$

Successfully initiated stopping with skip OS shutdown of i-08511bf0196c52a3f

Instances (7) Info Last updated less than a minute ago Connect Instance state Actions Launch instances

Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Avail
☐	Flask-Backend-Server	i-0c7532ed1e41586d5	Running	t3.micro	3/3 checks passed	View alarms	ap-s
☐	Express-Frontend-Server	i-06baafbb317193147	Running	t3.micro	3/3 checks passed	View alarms	ap-s

Get Public IPs

CMD : Terraform output

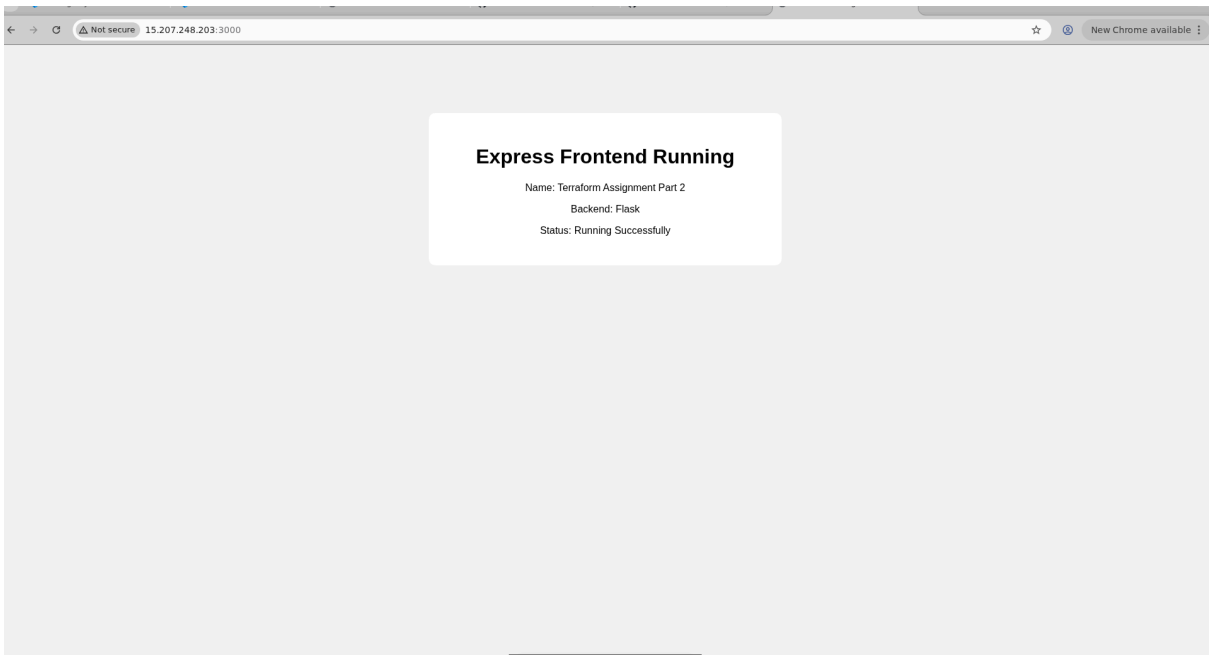
```

flask_public_ip = "3.108.219.115"
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$ terraform output
express_public_ip = "15.207.248.203"
flask_public_ip = "3.108.219.115"
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-2$

```


OUTPUT:

FRONTEND :



BACKEND :



Part 3: Deploy Flask and Express Using Docker and AWS Services

1. **Objective:** Deploy Flask and Express as **Docker containers** using **AWS ECR, ECS,** and **VPC** with Terraform.
2. **Steps:**
 - **ECR:**
 - Use Terraform to create two Elastic Container Registry (ECR) repositories: one for the Flask backend and one for the Express frontend.
 - Build Docker images for both applications and push them to their respective ECR repositories.
 - **VPC:**
 - Use Terraform to create a VPC with subnets, route tables, and security groups.
 - **ECS:**
 - Use Terraform to set up an ECS cluster.
 - Create two ECS services:
 - One for the Flask backend.
 - One for the Express frontend.
 - Use ECS Fargate or EC2 launch type to deploy the containers.
 - **Load Balancer:**
 - Use Terraform to provision an Application Load Balancer (ALB).
 - Configure ALB listeners to route requests to the appropriate ECS service.
3. **Expected Deliverables:**
 - Terraform configuration files for ECR, ECS, VPC, and ALB setup.
 - Docker images pushed to ECR.
 - ECS services running and accessible via the ALB.

General Requirements

- Ensure that all Terraform configurations follow best practices, including the use of `variables.tf` and `outputs.tf`.
- Use **Terraform State** (e.g., S3 backend) for state management.
- Use `terraform plan` and `terraform apply` to validate and deploy infrastructure.
- Document the deployment process in a `README.md` file.

CODE IS PLACED IN GIT LOCATION URL:

<https://github.com/UmeshChandra2001/terraform-ass-part-3>

PROJECT STRUCTURE:

terraform-ass-part-3/

```
| — terraform/  
|   | — provider.tf  
|   | — variables.tf  
|   | — terraform.tfvars  
|   | — backend.tf  
|   | — vpc.tf  
|   | — security-groups.tf  
|   | — ecr.tf  
|   | — ecs.tf  
|   | — alb.tf  
|   | — iam.tf  
|   | — outputs.tf  
|  
| — flask-backend/  
|   | — app.py  
|   | — requirements.txt  
|   | — Dockerfile  
|  
| — express-frontend/  
|   | — server.js  
|   | — package.json  
|   | — Dockerfile  
|  
| — README.md
```

Docker Build & Push Commands

Login to AWS ECR

CMD : aws configure

CMD : aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com

Initialize Terraform

CMD : terraform init

```
}
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/terraform$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/terraform$ terraform plan
```

CMD : terraform plan

CMD : terraform apply -auto-approve

```

+ arn = (known after apply)
+ arn_without_revision = (known after apply)
+ container_definitions = jsonencode(
  [
    {
      + essential = true
      + image = "467024768342.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest"
      + name = "flask-container"
      + portMappings = [
        {
          + containerPort = 5000
          + hostPort = 5000
        },
      ],
    },
  ],
)
+ cpu = "256"
+ enable_fault_injection = (known after apply)
+ execution_role_arn = "arn:aws:iam::467024768342:role/ecsTaskExecutionRole"
+ family = "flask-task"
+ id = (known after apply)
+ memory = "512"
+ network_mode = "awsvpc"
+ requires_compatibility = [
  + "FARGATE",
]
+ revision = (known after apply)
+ skip_destroy = false
+ tags_all = (known after apply)
+ track_latest = false
}

Plan: 4 to add, 0 to change, 0 to destroy.
aws_ecs_task_definition.express_task: Creating...
aws_ecs_task_definition.flask_task: Creating...
aws_ecs_task_definition.express_task: Creation complete after 0s [id=express-task]
aws_ecs_task_definition.flask_task: Creation complete after 0s [id=flask-task]
aws_ecs_service.express_service: Creating...
aws_ecs_service.flask_service: Creating...
aws_ecs_service.flask_service: Creation complete after 1s [id=arn:aws:ecs:ap-south-1:467024768342:service/main-cluster/flask-service]
aws_ecs_service.express_service: Creation complete after 1s [id=arn:aws:ecs:ap-south-1:467024768342:service/main-cluster/express-service]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
alb_dns_name = "main-alb-1968318114.ap-south-1.elb.amazonaws.com"
express_ecr_url = "467024768342.dkr.ecr.ap-south-1.amazonaws.com/express-frontend"
flask_ecr_url = "467024768342.dkr.ecr.ap-south-1.amazonaws.com/flask-backend"
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/terraform$

```

Build Flask Docker Image

`docker build -t flask-backend .`

`docker tag flask-backend:latest`

`<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest`

`docker push`

`<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest`

```

Downloading werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
Collecting packaging (from gunicorn>=7 requirements.txt (line 2))
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading gunicorn-26.0.0-py3-none-any.whl (103.4 kB)
 103.4/103.4 kB 78.7 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
 110.5/110.5 kB 535.8 MB/s eta 0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
 134.9/134.9 kB 541.2 MB/s eta 0:00:00
Downloading MarkupSafe-3.0.3-cp311-cp311-manylinux2014_x86_64.manylinux2_17_x86_64.manylinux2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.0-py3-none-any.whl (226 kB)
 226.5/226.5 kB 547.6 MB/s eta 0:00:00
Downloading packaging-26.2-py3-none-any.whl (100 kB)
 100.2/100.2 kB 527.2 MB/s eta 0:00:00
Installing collected packages: packaging, markupsafe, itsdangerous, click, blinker, werkzeug, Jinja2, gunicorn, flask
Successfully installed blinker-1.9.0 click-8.3.3 flask-3.1.3 gunicorn-26.0.0 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 packaging-26.2 werkzeug-3.1.8
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 24.0 -> 26.1.1
[notice] To update, run: pip install --upgrade pip
--> Removed intermediate container 0a0c946ad3d1
--> 28a737bd6191
Step 5/7 : COPY . .
--> 04013bfff4b0
Step 6/7 : EXPOSE 5000
--> Running in e482044fd0d6
--> Removed intermediate container e482044fd0d6
--> 02f06905461c
Step 7/7 : CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
--> Running in 9ca423303906
--> Removed intermediate container 9ca423303906
--> 2d24910cfbad
Successfully built 2d24910cfbad
Successfully tagged flask-backend:latest
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/flask-backend$ docker tag flask-backend:latest 467024768342.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/flask-backend$ docker push 467024768342.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
The push refers to repository [467024768342.dkr.ecr.ap-south-1.amazonaws.com/flask-backend]
9083c0213931: Pushed
1c903da26a44: Pushed
9ca230ea3c4e: Pushed
02acacc0a91b: Pushed
94e760ec3074: Pushed
7999dc77eb30: Pushed
2e53cb234c59: Pushed
79dd1f4c855c: Pushed
latest: digest: sha256:944bf56a2ef35b276871f2bcb4c485aa3499a63714e173853e71363cd5710c1 size: 1990
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/flask-backend$

```

Build Express Docker Image

`docker build -t express-frontend .`

`docker tag express-frontend:latest`

`<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest`

`docker push`

`<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest`

```
---> d67555520110
Step 3/7 : COPY package.json .
---> 332cc2c24868
Step 4/7 : RUN npm install
---> Running in ea83a7acde9f

added 69 packages, and audited 70 packages in 4s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.14.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.14.1
npm notice To update run: npm install -g npm@11.14.1
npm notice
---> Removed intermediate container ea83a7acde9f
---> c775b2c38199
Step 5/7 : COPY . .
---> ee9b0e6c4b7d
Step 6/7 : EXPOSE 3000
---> Running in fc6cfd3556b0
---> Removed intermediate container fc6cfd3556b0
---> 442f2ff5a29b
Step 7/7 : CMD ["npm", "start"]
---> Running in e0a0aed34f39
---> Removed intermediate container e0a0aed34f39
---> 1d81be424764
Successfully built 1d81be424764
Successfully tagged express-frontend:latest
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/express-frontend$ docker tag express-frontend:latest <467024768342>.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:lates
t
bash: <467024768342>.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest: No such file or directory
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/express-frontend$ docker tag express-frontend:latest 467024768342.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/express-frontend$ docker push 467024768342.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
The push refers to repository [467024768342.dkr.ecr.ap-south-1.amazonaws.com/express-frontend]
8ac514c22e18: Pushed
6f4845368112: Pushed
a14ff8a555eb: Pushed
5c17b546837b: Pushed
afa543f85b46: Pushed
e3b358715ead: Pushed
4983b92ae796: Pushed
29df493baa13: Pushed
latest: digest: sha256:7abbb6ab007bd09aa524fa5324592f9a99463e2f9ab0652abdcdf3181e5fb9d54 size: 1989
ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/express-frontend$
```

Update ECS Services

`aws ecs update-service \`

`--cluster main-cluster \`

`--service flask-service \`

`--force-new-deployment \`

`--region ap-south-1`

```

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/flask-backend$ aws ecs update-service \
--cluster main-cluster \
--service flask-service \
--force-new-deployment \
--region ap-south-1
{
  "service": {
    "serviceArn": "arn:aws:ecs:ap-south-1:467024768342:service/main-cluster/flask-service",
    "serviceName": "flask-service",
    "clusterArn": "arn:aws:ecs:ap-south-1:467024768342:cluster/main-cluster",
    "loadBalancers": [
      {
        "targetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:467024768342:targetgroup/flask-tg/25fb67ccd9c43918",
        "containerName": "flask-container",
        "containerPort": 5000
      }
    ],
    "serviceRegistries": [],
    "status": "ACTIVE",
    "desiredCount": 1,
    "runningCount": 1,
    "pendingCount": 0,
    "launchType": "FARGATE",
    "platformVersion": "LATEST",
    "platformFamily": "Linux",
    "taskDefinition": "arn:aws:ecs:ap-south-1:467024768342:task-definition/flask-task:1",
    "deploymentConfiguration": {
      "deploymentCircuitBreaker": {
        "enable": false,
        "rollback": false
      },
      "maximumPercent": 200,
      "minimumHealthyPercent": 100
    },
    "deployments": [
      {
        "id": "ecs-svc/9907402190169747011",
        "status": "PRIMARY",
        "taskDefinition": "arn:aws:ecs:ap-south-1:467024768342:task-definition/flask-task:1",
        "desiredCount": 0,
        "pendingCount": 0,
        "runningCount": 0,
        "failedTasks": 0,
        "createdAt": 1778435291.043,
        "updatedAt": 1778435291.043,
        "launchType": "FARGATE"
      }
    ]
  }
}

```

```

aws ecs update-service \
--cluster main-cluster \
--service express-service \
--force-new-deployment \
--region ap-south-1

```

```

ubuntu@ip-172-31-16-42:~/TuteDude-Assignments/terraform-ass-part-3/flask-backend$ aws ecs update-service \
--cluster main-cluster \
--service express-service \
--force-new-deployment \
--region ap-south-1
{
  "service": {
    "serviceArn": "arn:aws:ecs:ap-south-1:467024768342:service/main-cluster/express-service",
    "serviceName": "express-service",
    "clusterArn": "arn:aws:ecs:ap-south-1:467024768342:cluster/main-cluster",
    "loadBalancers": [
      {
        "targetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:467024768342:targetgroup/express-tg/7776da8dcd3157a9",
        "containerName": "express-container",
        "containerPort": 3000
      }
    ],
    "serviceRegistries": [],
    "status": "ACTIVE",
    "desiredCount": 1,
    "runningCount": 1,
    "pendingCount": 0,
    "launchType": "FARGATE",
    "platformVersion": "LATEST",
    "platformFamily": "Linux",
    "taskDefinition": "arn:aws:ecs:ap-south-1:467024768342:task-definition/express-task:1",
    "deploymentConfiguration": {
      "deploymentCircuitBreaker": {
        "enable": false,
        "rollback": false
      },
      "maximumPercent": 200,
      "minimumHealthyPercent": 100
    },
    "deployments": [
      {
        "id": "ecs-svc/8579091229070958087",
        "status": "PRIMARY",
        "taskDefinition": "arn:aws:ecs:ap-south-1:467024768342:task-definition/express-task:1",
        "desiredCount": 0,
        "pendingCount": 0,
        "runningCount": 0,
        "failedTasks": 0,
        "createdAt": 1778435317.186,
        "updatedAt": 1778435317.186,
        "launchType": "FARGATE"
      }
    ]
  }
}

```

OUTPUT:

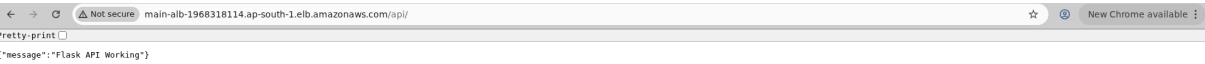
FRONTEND:

<http://main-alb-1968318114.ap-south-1.elb.amazonaws.com/>



BACKEND:

<http://main-alb-1968318114.ap-south-1.elb.amazonaws.com/api/>



Submission Guidelines :- Attach Screenshots or command along with explanation and submit in doc(google doc or microsoft doc) format and share your github repo link